

RTL Design Sherpa

Binary BCH Codec Micro-Architecture Specification 0.1 (draft)

October 3, 2026

Table of Contents

1 Bch Mas Index....9

2 Document Information....10

2.1 Binary BCH Codec Micro-Architecture Specification....10

2.2 Revision History....11

2.3 Document Purpose....12

2.4 Intended Audience....13

2.5 Related Documents....14

3 Methodology....15

3.1 HAS to MAS relationship....15

3.2 FSM policy....16

3.3 Prove-by-design / kmap philosophy....17

3.4 Carrying open PRD decisions....18

4 Block Inventory....19

5 Encoder Core (bch_encoder_core)....21

5.1 Purpose....22

5.1.1 Figure 2.1: Encoder core block diagram....22

5.2 Parameters....23

5.3 Interface....24

5.4 Microarchitecture internals....26

5.4.1 Beat-parallel LFSR parity generator....26

5.4.2 Parity mux....26

5.4.3 Output skid....26

5.5 FSM policy....27
5.6 Timing....28
5.7 Notes....29
6 Syndrome Unit (bch_syndrome_unit)....30
6.1 Purpose....31
6.1.1 Figure 2.2: Syndrome unit block diagram....31
6.2 Parameters....32
6.3 Interface....33
6.4 Microarchitecture internals....35
6.4.1 Odd-syndrome recurrence....35
6.4.2 All-zero bypass qualifier....35
6.4.3 Completion qualifier....35
6.5 FSM policy....36
6.6 Timing....37
6.7 Notes....38
7 Key Equation Solver (bch_key_equation_solver)....39
7.1 Purpose....40
7.2 Parameters....41
7.3 Interface....42
7.4 Candidate algorithms....43
7.5 Microarchitecture internals....45
7.5.1 "RIBM" and "EUCLID" — fixed-iteration arrays....45
7.5.2 "SMALL_T" — conditional minimal control FSM....45
7.6 FSM policy....46

7.7 Timing....47
7.8 Notes....48
8 Chien Search (bch_chien_search)....49
8.1 Purpose....50
8.1.1 Figure 2.3: Chien search block diagram....50
8.2 Parameters....51
8.3 Interface....52
8.4 Microarchitecture internals....53
8.4.1 Parallel Horner cells....53
8.4.2 Direct bit-flip accumulation....53
8.4.3 Root count (the verdict lives in the decoder core)....53
8.5 FSM policy....54
8.6 Timing....55
8.7 Notes....56
9 Decoder Core Integration (bch_decoder_core)....57
9.1 Purpose....58
9.1.1 Figure 2.4: Decoder core block diagram....59
9.2 Parameters....60
9.3 Interface....62
9.4 Microarchitecture internals....63
9.4.1 Block buffer....63
9.4.2 Stage sequencer....63
9.4.3 Corrected-stream re-check....63
9.4.4 Correctability verdict....63

9.5 FSM policy..... 64
9.6 Timing..... 65
9.7 Notes..... 66
10 Core Signals..... 67
10.1 Reset strategy..... 68
10.2 bch_encoder_core signal contract..... 69
10.3 bch_decoder_core signal contract..... 72
10.4 Timing rules..... 75
11 Signal Contracts and K-maps..... 76
11.1 What a contract is..... 76
11.2 Which signals have contracts..... 77
11.3 Pre-RTL citation posture..... 78
11.4 Regenerating the workbook..... 79
11.5 Workbook contents..... 80

List of Figures

Figure 2.1: Encoder core block diagram.....22
Figure 2.2: Syndrome unit block diagram.....31
Figure 2.3: Chien search block diagram.....50
Figure 2.4: Decoder core block diagram.....58

List of Tables

Table 1: Table 0.1: Related documents....14
Table 2: Table 1.1: MAS block inventory....19
Table 3: Table 1.2: Imported GF primitives....20
Table 4: Table 2.1: Encoder core parameters....23
Table 5: Table 2.2: Encoder core ports....24
Table 6: Table 2.3: Syndrome unit parameters....32
Table 7: Table 2.4: Syndrome unit ports....33
Table 8: Table 2.5: Key-equation solver parameters....41
Table 9: Table 2.6: Key-equation solver ports....42
Table 10: Table 2.7: Key-equation solver candidates....43
Table 11: Table 2.8: Chien search parameters....51
Table 12: Table 2.9: Chien search ports....52
Table 13: Table 2.10: Decoder core parameters....59
Table 14: Table 2.11: Decoder core ports....61
Table 15: Table 3.1: bch_encoder_core signal contract....68
Table 16: Table 3.2: bch_decoder_core signal contract....71

List of Waveforms

No waveforms in this document.

1 Bch Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Binary BCH Codec Micro-Architecture Specification

Property	Value
Document Title	Binary BCH Codec Micro-Architecture Specification
Version	0.2 (draft)
Date	October 3, 2026
Status	Draft — no RTL exists; every structural choice is either carried from the PRD or open and tied to a PRD decision ID
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.1	2026-10-03	RTL Design Sherpa	First draft, written before any RTL exists. Carries the HAS architecture one level down: per-block signal-level intent, FSM policy, timing, and signal-contract references. Numbers are analytic placeholders derived from parameters; none are measured.
0.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.

2.3 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level view of the Binary BCH codec — the HAS (`../bch_has/`) taken one level down. It is the document an RTL author reads before writing SystemVerilog and a verification engineer reads before writing assertions.

It covers:

- each functional block's internal structure, signals, and parameters
- the FSM policy for every block: where a state machine is forbidden, where one minimal control FSM is permitted, and what replaces FSMs elsewhere
- cycle-level timing and throughput placeholders tied to the open PRD decisions
- the signal-contract posture that will be checked by the generated kmap workbook

2.4 Intended Audience

- RTL implementers writing the encoder, decoder, and GF-datapath blocks
- Verification engineers checking the golden-model equivalence and the formal properties in `formal/`
- Architects reviewing whether a candidate D6 throughput or D11 solver choice fits a consumer's area/latency target

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Product Requirements	`../../PRD.md`	the decision table (D1-D12) and the candidate profiles
Hardware Architecture Specification	`../bch_has/bch_has_index.md`	high-level block diagram, data flow, solver options, interfaces, parameters
Signal-contract workbook	`../../bch_signal_contracts.xlsx`	generated contract sheets and kmap decision tables
Signal-contract generator	`../../gen_bch_signal_contracts_kmaps.py`	Python generator for the workbook
References	`../../References/README.md`	the standards and papers, with source and licence
CLAUDE.md	`../../CLAUDE.md`	area facts for a session working here
Handbook	`vault/handbook/design/`	streaming-no-fsm, minimal-fsm, signal-contracts-and-kmaps, valid-ready contracts

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Methodology

3.1 HAS to MAS relationship

The Hardware Architecture Specification (`../bch_has/`) records what the BCH codec is, where it sits, what it presents at its boundaries, and what parameters configure it. This Micro-Architecture Specification carries that architecture one level down: per-block signal tables, cycle behavior, FSM policy, and the signal-contract references that became the contract workbook cited against landed RTL (`ch04_contracts/01_signal_contracts.md`).

The rule in this repo is one source per fact. The MAS does not restate the PRD decision table or the HAS block diagram; it cross-references them and adds the implementation detail that belongs at this level. Parameters and throughput numbers that are not yet fixed are marked **TBD** and tied to the PRD decision ID that fixes them, following the convention in `../bch_has/ch06_integration/02_parameters.md`.

3.2 FSM policy

The repo's datapath FSM rule, taken from the design handbook, is:

> **No FSM in the per-beat data path. Not a minimal one, not a two-state one.** FSMs belong to control: descriptor lifecycle, schedulers, init sequencers, error recovery. If a state machine can observe a data beat, it is in the wrong place. — `vault/handbook/design/streaming-no-fsm.md`

What replaces a datapath FSM in this block:

Instead of a state that…	Write
counts beats, or waits N cycles	a counter plus a comparator
remembers "I am mid-burst"	a `r_valid` flag on the pipeline stage
serializes because a resource is shared	an arbiter and a qualifier, not a turn-taking machine
gates until a condition holds	a qualifier ANDed into the handshake

Where a state machine is genuinely justified, the handbook requires:

> Fewest states that carry the real distinctions. A state that only waits one fixed cycle, or that differs from a sibling only by a datapath value, is a register, not a state. Merge states whose outputs and successors are identical. Idiom: `typedef enum logic [N-1:0] {...} state_t;` + two-process form - registered `r_state`, combinational `w_next_state` with a default-hold first line and a default arm. One FSM per module; nested/communicating FSMs are a smell that the block wants splitting. Outputs: prefer registered (Moore) at module boundaries; combinational decode of `r_state` internally is fine but belongs in the K-maps. — `vault/handbook/design/minimal-fsm.md`

Every block page in this chapter states its FSM policy explicitly and cites this page. Blocks that are purely datapath carry no FSM; the one block that sequences decoder stages carries exactly one minimal control FSM.

3.3 Prove-by-design / kmap philosophy

The signal-contract and kmap discipline in this repo treats the act of mirroring RTL expressions in a computed workbook as a design review in itself:

> The act of mirroring expressions IS a design review. The stream workbook's first generation surfaced six findings the test suite had not…
> — `vault/handbook/design/signal-contracts-and-kmaps.md`

For the BCH MAS the posture was adopted pre-RTL (the contracts first cited the MAS pages verbatim) and completed post-RTL: on 2026-10-08 (TASK-003 close) every citation in `bch_signal_contracts.xlsx` moved to the landed RTL `file:line`, where `verify_citations` gates the build against drift. The workbook is generated by `gen_bch_signal_contracts_kmaps.py`; it is never hand-edited.

3.4 Carrying open PRD decisions

Every undecided item in the PRD is carried as **TBD** with the decision ID that resolves it. Examples in this document:

Parameter / choice	PRD decision
field dimension `m`	D1
correctable bits `t`	D2
block length `N_BITS`	D3
encoder/decoder build mix	D4
erasure support	D5
throughput architecture and cycle counts	D6
bits per beat `BITS_PER_BEAT`	D9
key-equation solver algorithm	D11

No numeric value is invented. Cycle-count formulas are stated in terms of the parameters above (for example, `N_BITS` cycles per syndrome pass in the serial case), and throughput trade-offs are placeholders tied to D6.

4 Block Inventory

The MAS documents the target blocks below. Each page follows the FUB-page skeleton from the pumice MAS: header metadata, purpose, parameters, interface, microarch internals, timing, and notes. The GF(2^m) primitives are imported from the reed-solomon component per PRD D7 and are not re-documented here.

Table 1.1: MAS block inventory

Block	RTL target	Role	MAS page
`bch_encoder_core`	`rtl/bch_encoder_core.sv`	Systematic encoder: bit serialiser, GF LFSR parity generator, parity mux, bit packer	01_encoder.md
`bch_syndrome_unit`	`rtl/bch_syndrome_unit.sv`	Computes the `t` odd syndromes `S_1 ... S_2t-1` over the serial bit stream using `t` parallel GF MAC lanes	02_syndrome_unit.md
`bch_key_equation_solver`	`rtl/bch_key_equation_solver.sv`	Inverts the key equation from odd syndromes to error-locator `Lambda(x)`; D11 selects riBM, Euclid, or step-by-step/PGZ	03_key_equation_solver.md
`bch_chien_search`	`rtl/bch_chien_search.sv`	Evaluates `Lambda(x)` at all `N_BITS` bit positions, flags roots, flips bits; no Forney stage	04_chien_search.md
`bch_decoder_core`	`rtl/bch_decoder_core`	Block buffer + stage sequencer	05_decoder_core.

Block	RTL target	Role	MAS page
	`.sv`	between syndrome, solver, Chien, and corrector; exactly one minimal control FSM	md

The GF layer is imported, not duplicated:

Table 1.2: Imported GF primitives

Primitive	Source in `projects/components/ecc-ip/reed-solomon/rtl/fub/gf/`	Used by
`gf_pkg`	`gf_pkg.sv`	all blocks (tables, constants, generator polynomial)
`gf_mul`	`gf_mul.sv`	syndrome unit, key-equation solver, Chien search
`gf_mul_const`	`gf_mul_const.sv`	LFSR encoder taps, Chien Horner cells
`gf_inv`	`gf_inv.sv`	Euclidean solver only (if D11 selects EUCLID)

The adapter blocks (`axis_adapter`, `axi4_job_adapter`, register block) are specified in `../bch_has/ch04_interfaces/` and are not taken to the signal level in this MAS.

5 Encoder Core (bch_encoder_core)

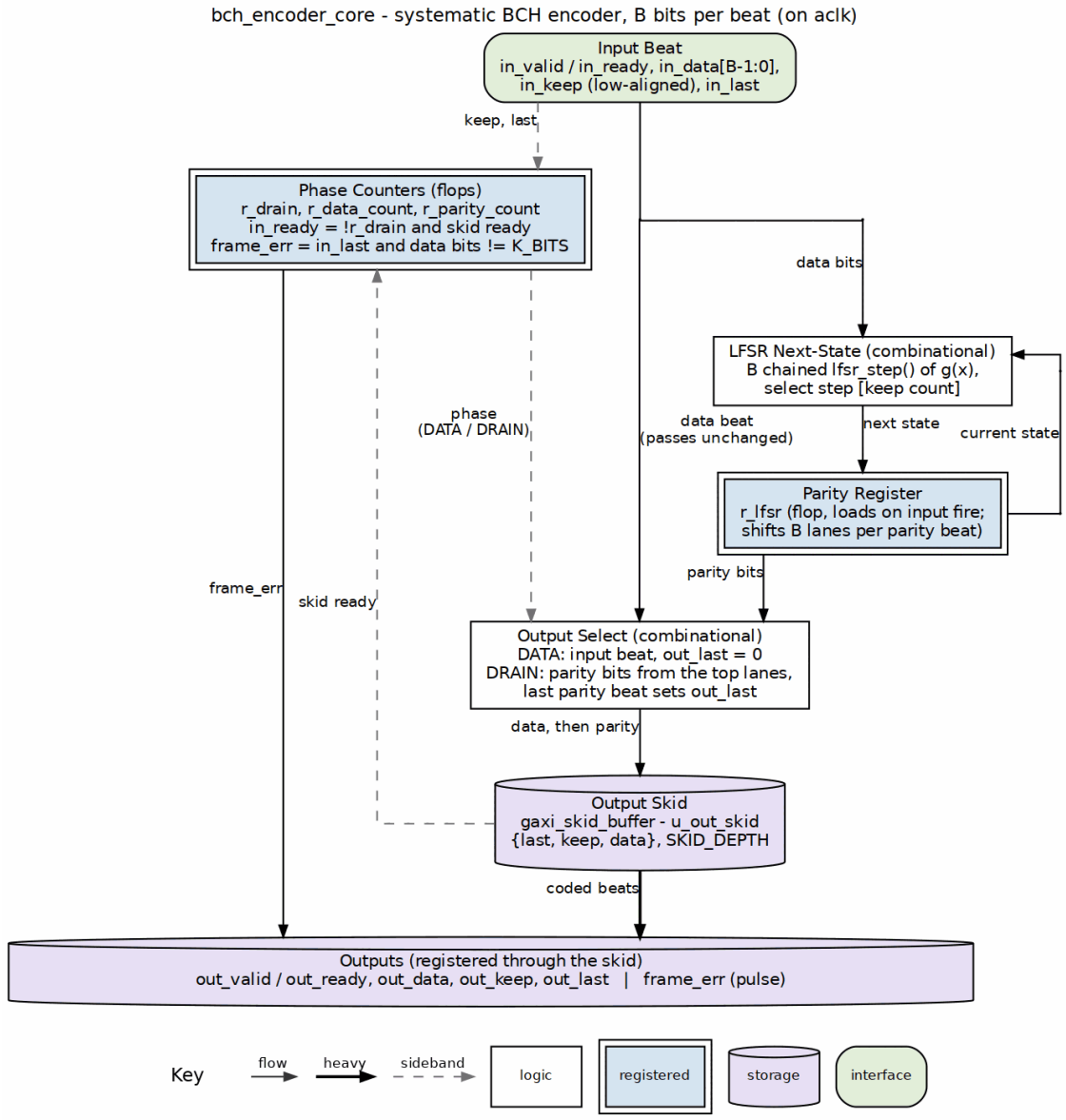
Module: bch_encoder_core.sv **Location:** rtl/macro/ **Category:** streaming datapath **Parent:** bch_top / standalone encoder wrapper **Status:** landed — rtl/macro/bch_encoder_core.sv, gate DV green on the three standing profiles

---

5.1 Purpose

`bch_encoder_core` systematically encodes a `K_BITS`-bit data block into an `N_BITS`-bit BCH codeword. Data bits pass through unchanged; the block then appends the `N_BITS - K_BITS` parity bits produced by a GF(2^m) LFSR whose taps come from the generator polynomial computed by `gf_pkg`. The output stream is longer than the input stream, so the encoder is not a pure back-to-back pipeline: it holds `in_ready` low while parity drains.

5.1.1 Figure 2.1: Encoder core block diagram



Encoder core block diagram

5.2 Parameters

Table 2.1: Encoder core parameters

Parameter	Type	Range	Default	Meaning	PRD
FIELD_DIM	int	3..16	**TBD**	m; field is GF(2^m)	D1
T_BITS	int	1..(2^m-1)/2	**TBD**	correctable bit errors	D2
N_BITS	int	2t+1 .. 2^m-1	**TBD**	codeword length in bits	D3
BITS_PER_BEAT	int	1 .. K_BITS	**TBD**	bits per valid/ready beat	D9

Derived: K_BITS = N_BITS - DEGREE_G, where DEGREE_G is the degree of the generator polynomial computed by gf_pkg from m, t, and the profile parameters. DEGREE_G is at most m * T_BITS and equals N_BITS - K_BITS.

5.3 Interface

Table 2.2: Encoder core ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock for the whole core
`aresetn`	in	1	active-low synchronous reset
`in_valid`	in	1	a beat of data bits is offered
`in_ready`	out	1	the core takes it this cycle
`in_data`	in	`BITS_PER_BEAT`	data bits, bit 0 in the low bit; partial final beat uses `in_keep`
`in_keep`	in	`BITS_PER_BEAT`	present-bit mask, low-aligned; partial only on the block's final beat
`in_last`	in	1	this beat carries the `K_BITS`-th data bit
`out_valid`	out	1	a beat of coded bits is offered
`out_ready`	in	1	the consumer takes it
`out_data`	out	`BITS_PER_BEAT`	data bits while `!w_parity_phase`, parity bits while `w_parity_phase`
`out_keep`	out	`BITS_PER_BEAT`	present-bit mask; partial at the end of data and at the end of parity
`out_last`	out	1	this beat carries

Signal	Direction	Width	Description
			the N_BITS-th coded bit
frame_err	out	1	pulse: a block ended with other than K_BITS data bits

5.4 Microarchitecture internals

5.4.1 Beat-parallel LFSR parity generator

The core works on whole B-bit beats; there is no serialiser. The parity register `r_lfsr` holds deg(g) lanes of m bits over GF(2^m), with the generator polynomial's coefficients (computed by `bch_pkg` from `FIELD_DIM`, `PRIM_POLY`, `T_BITS` and `FIRST_ROOT`) on its taps. Its next-state logic is a chain of B `lfsr_step()` evaluations, one per bit of the beat (bit 0 first), built as an unpacked array `w_chain[0..B]`; the keep count of the beat selects `w_chain[keep count]`, so a partial final data beat advances the register by exactly the bits present. The multiplies use `gf_mul_fn` from the imported `gf_pkg`; no `gf_lfsr_encoder` instance is used.

The register loads on every accepted data beat. During the parity drain it shifts left by one beat's worth of lanes each emitted beat, so it is all-zero again when the drain ends and the next block needs no explicit clear.

5.4.2 Parity mux

The output mux selects data bits while the block is in the data phase and parity bits once the data phase completes:

```
w_parity_phase = (r_bit_count >= K_BITS) && w_beat_is_parity
```

`w_beat_is_parity` is true whenever the current output position is inside the parity region; it is derived from the same bit counter:

```
w_beat_is_parity = (r_bit_count + BITS_PER_BEAT - 1) >= K_BITS
```

The mux output is `out_data = w_parity_phase ? w_parity_bits : w_data_bits`.

5.4.3 Output skid

There is no bit packer. `{last, keep, data}` from the output select goes into one `gaxi_skid_buffer` (`u_out_skid`, depth `SKID_DEPTH`, 2..8), whose read side is the output port. `out_keep` is full except on the final data beat and the final parity beat, where it marks the partial beat. `out_last` raises on the beat that carries the `N_BITS`-th coded bit:

```
w_enc_out_last = w_parity_phase &&
                 (r_parity_count + w_par_beat_len - 1 == N_BITS -
K_BITS - 1)
```

5.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). The sequencing that another design might put in a state machine is replaced by:

- `r_bit_count`: counts input/output bit position in the block, 0 .. `N_BITS-1`
- `r_parity_count`: counts parity bits emitted, 0 .. `N_BITS-K_BITS-1`
- `r_drain`: set by the accepted `in_last` beat, cleared by the last parity beat
- `w_parity_phase`: combinational qualifier selecting parity vs data
- `w_last`: combinational qualifier marking the final output beat

`in_ready` is combinational: `!r_drain && w_skid_wr_ready`, the output skid's write-side ready qualified by the drain flag.

5.6 Timing

- One input beat is accepted per cycle while `in_ready` is high.
- The data phase is `ceil(K_BITS / BITS_PER_BEAT)` beats.
- The parity phase is `ceil((N_BITS - K_BITS) / BITS_PER_BEAT)` beats.
- `in_ready` is held low during the parity phase; back-to-back blocks need an upstream FIFO sized for at least `N_BITS - K_BITS` bits.
- The exact blocks-per-cycle numbers are placeholders tied to PRD D6 and D9.

5.7 Notes

- **Wrong-length block handling:** If `in_last` arrives before or after the `K_BITS`-th data bit, `frame_err` pulses and the block is still encoded as given (systematic data pass-through with parity appended to whatever data arrived). This is the encoder-side analog of the decoder's `out_status_frame_err` (R1/R2):

  ```
  w_frame_err = in_last && (r_data_count + keep_count != K_BITS)
  ```

- **OFF-state testing convention:** For tests that need to verify the core tolerates idle cycles, the LFSR drains to zero with the last parity beat and the output skid stalls cleanly when `out_ready` is low.

- The LFSR is configured by the same `FIELD_DIM`, `T_BITS`, `N_BITS`, `PRIM_POLY`, and `FIRST_ROOT` parameters as the rest of the codec; its generator polynomial comes from `bch_pkg` (PRD D3/D8).

6 Syndrome Unit (bch_syndrome_unit)

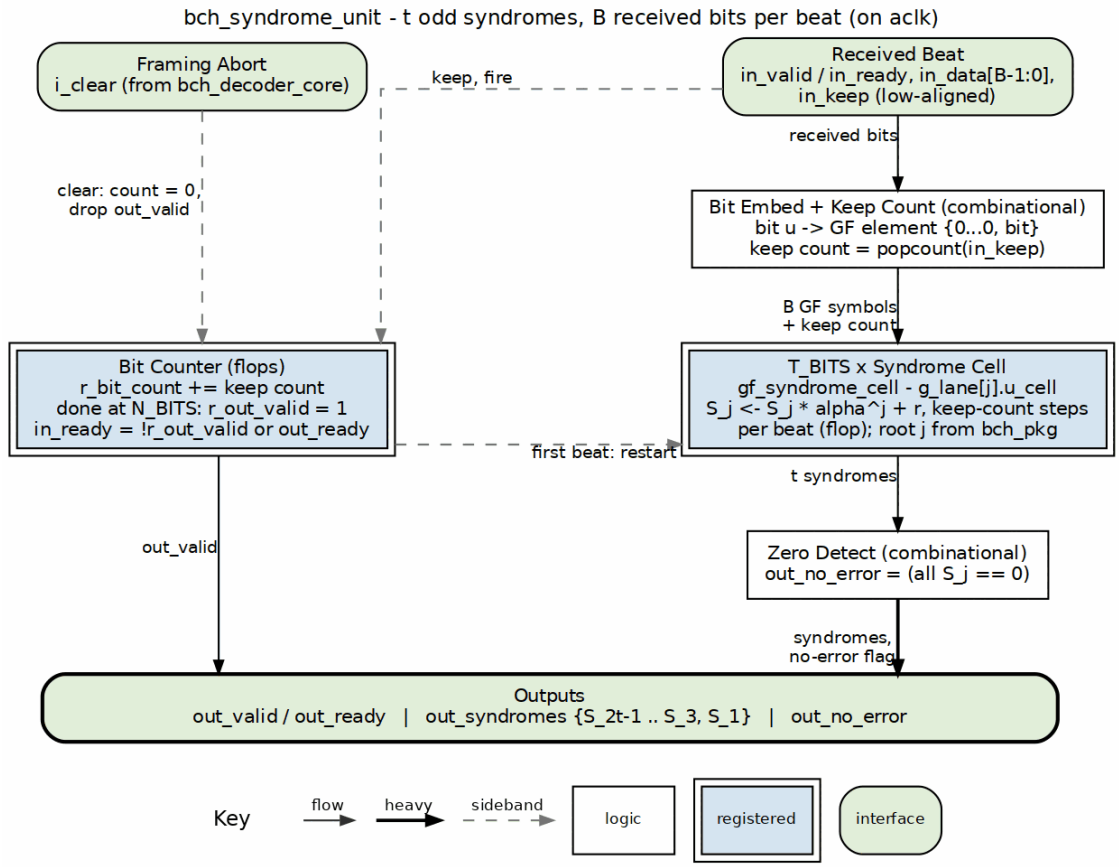
Module: bch_syndrome_unit.sv **Location:** rtl/fub/ **Category:** streaming datapath **Parent:** bch_decoder_core **Status:** landed — rtl/fub/bch_syndrome_unit.sv, gate DV green on the three standing profiles

---

6.1 Purpose

`bch_syndrome_unit` computes the `t` odd syndromes `S_1, S_3, ..., S_2t-1` of the received BCH codeword as the bits arrive. The even syndromes are not computed directly; they are recovered from the odd ones through the binary BCH evenness shortcut `S_2j = S_j^2` (PRD §2). The unit contains `t` parallel GF(2^m) multiply-accumulate lanes, one per odd syndrome.

6.1.1 Figure 2.2: Syndrome unit block diagram



Syndrome unit block diagram

6.2 Parameters

Table 2.3: Syndrome unit parameters

Parameter	Type	Range	Default	Meaning	PRD
FIELD_DIM	int	3..16	**TBD**	m; field is GF(2^m)	D1
T_BITS	int	1..(2^m-1)/2	**TBD**	correctable bit errors	D2
N_BITS	int	2t+1 .. 2^m-1	**TBD**	codeword length in bits	D3
BITS_PER_BEAT	int	1 .. K_BITS	**TBD**	bits per valid/ready beat	D9

6.3 Interface

Table 2.4: Syndrome unit ports

Signal	Direction	Width	Description
aclk	in	1	one clock
aresetn	in	1	active-low synchronous reset
in_valid	in	1	received bits are offered
in_ready	out	1	unit takes them this cycle
in_data	in	BITS_PER_BEAT	received bits, low-aligned
in_keep	in	BITS_PER_BEAT	present-bit mask for partial final beat
in_last	in	1	this beat carries the N_BITS-th received bit
out_valid	out	1	t odd syndromes are valid
out_ready	in	1	downstream accepts the syndromes
out_syndromes	out	T_BITS * m	{S_2t-1, ..., S_3, S_1} packed GF values
out_no_error	out	1	all odd syndromes are zero
i_clear	in	1	abort the in-flight accumulation: bit counter and any pending out_valid drop, and the next

Signal	Direction	Width	Description
			accepted beat starts a fresh block. Asserted by the decoder core on a framing violation (a block past `N_BITS`, issue #90) so the garbage count cannot leak into the next block

6.4 Microarchitecture internals

6.4.1 Odd-syndrome recurrence

For a received polynomial `r(x) = sum_{i=0}^{N_BITS-1} r_i x^i`, the odd syndromes are:

```
S_j = r(alpha^j) = sum_i r_i * (alpha^j)^i   for j = 1, 3, ..., 2t-1
```

As bits arrive one at a time, each lane accumulates by Horner's rule:

```
S_j <- S_j * alpha^j + r_i
```

The multiplier by the constant `alpha^j` is a `gf_mul_const` instance (imported from `projects/components/ecc-ip/reed-solomon/rtl/fub/gf/gf_mul_const.sv`). No per-position power table is stored; the constant `alpha^j` is wired once per lane.

With `BITS_PER_BEAT = B > 1`, each lane performs `B` Horner steps per cycle:

```
S_j <- (((S_j * alpha^j) + r_i0) * alpha^j + r_i1) * alpha^j + ... +
r_iB-1
```

This is a short feed-forward chain of `B` constant multiplies and adds.

6.4.2 All-zero bypass qualifier

The no-error bypass is a combinational qualifier, not a state:

```
w_no_error = (S_1 == 0) && (S_3 == 0) && ... && (S_2t-1 == 0)
```

When `w_no_error` is true on the cycle `w_syndrome_done` raises, the decoder core skips the key-equation solver and Chien search and drains the block buffer unchanged.

6.4.3 Completion qualifier

The syndromes are complete once the last received bit has been accumulated:

```
w_syndrome_done = (r_bit_count == N_BITS - 1) && in_valid && in_ready
```

`out_valid` is a registered flag set by `w_syndrome_done` and cleared by the handshake with the downstream solver.

6.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). Control is reduced to:

- `r_bit_count`: counts received bits, 0 .. `N_BITS-1`
- `r_lane_valid`: registered output valid flag
- `w_syndrome_done`: combinational completion qualifier
- `w_no_error`: combinational all-zero qualifier

Each lane is a pure multiply-accumulate datapath; the lane count and the output valid flag replace any state machine.

6.6 Timing

- One received bit advances every lane by one Horner step.
- Serial case (`BITS_PER_BEAT = 1`): `N_BITS` cycles per block.
- Parallel-beat case: `ceil(N_BITS / BITS_PER_BEAT)` cycles per block, with `B` chained GF constant multiplies per lane per cycle.
- The exact cycle count is a placeholder tied to PRD D6.

6.7 Notes

- A shortened code (`N_BITS < 2^m - 1`) is handled by treating the missing leading bits as zero: the Horner recurrence naturally accumulates starting from `S_j = 0`, which is equivalent to zero-padding the high-order coefficients.
- The evenness shortcut means the formal proof targets include `S_2j == gf_square(S_j)` for every odd `j`; the square in GF(2^m) is a linear bit-permutation and is cheap to verify.
- `out_no_error` is a combinational decode of the syndrome registers; it is stable from `w_syndrome_done` until the output handshake clears it.

7 Key Equation Solver (bch_key_equation_solver)

Module: bch_key_equation_solver.sv **Location:** rtl/fub/ **Category:** datapath + conditional control FSM **Parent:** bch_decoder_core **Status:** landed — rtl/fub/bch_key_equation_solver.sv implements “RIBM” (the documented default when no consumer names a small-t profile); KES_ALGO values other than “RIBM” raise an elaboration error rather than silently falling back. PRD D11 stays open for EUCLID / SMALL_T.

7.1 Purpose

`bch_key_equation_solver` inverts the key equation from the `t` odd syndromes `S_1, S_3, ..., S_2t-1` to the error-locator polynomial `Lambda(x)`. The choice of algorithm is PRD D11; this page documents all three candidates and their microarchitectural implications. Nothing outside the solver knows which algorithm produced `Lambda`: the Chien search, block buffer, and corrector are identical in every build.

7.2 Parameters

Table 2.5: Key-equation solver parameters

Parameter	Type	Range	Default	Meaning	PRD
`FIELD_DIM`	int	3..16	**TBD**	`m`; field is GF(2^m)	D1
`T_BITS`	int	1..(2^m-1)/2	**TBD**	correctable bit errors	D2
`KES_ALGO`	string	`"RIBM"`, `"EUCLID"`, `"SMALL_T"`	**TBD**	solver algorithm	D11

7.3 Interface

Table 2.6: Key-equation solver ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock
`aresetn`	in	1	active-low synchronous reset
`in_valid`	in	1	odd syndromes are valid
`in_ready`	out	1	solver accepts syndromes
`in_syndromes`	in	`T_BITS * m`	`{S_2t-1, ..., S_3, S_1}` packed GF values
`out_valid`	out	1	`Lambda(x)` coefficients are valid
`out_ready`	in	1	downstream accepts `Lambda`
`out_lambda`	out	`(T_BITS + 1) * m`	locator coefficients `Lambda_0 .. Lambda_T_BITS`
`out_lambda_degree`	out	`$clog2(T_BITS + 1)`	actual degree of `Lambda` (clamped to `T_BITS` when `out_more_than_t`)
`out_more_than_t`	out	1	any `Lambda_i` above index `T_BITS` is nonzero — more than t errors; the decoder core folds it into the R2 verdict

7.4 Candidate algorithms

The trade-off table below is copied from `../bch_has/ch03_architecture/03_solver_options.md`.

Table 2.7: Key-equation solver candidates

	inversionless BM (`"RIBM"`)	**modified Euclidean (`"EUCLID"`)**	**step-by-step / PGZ (`"SMALL_T"`)**
Algorithm	reformulated inversionless Berlekamp-Massey, Sarwate and Shanbhag 2001; seeded with the t odd syndromes	Sugiyama's Euclidean algorithm with cross-multiplication in place of division (Shao et al. 1985)	Massey 1965 step-by-step correction, or the Peterson-Gorenstein-Zierler direct matrix solve for small t
Array	3t + 1 processing elements, 2 registers each	four register arrays: R and Q of 2t + 1 symbols, the shifted locator pair of 2t + 3	control-heavy but small datapath: one or a few GF multipliers and a syndrome update loop
GF multipliers	2 per element: 6t + 2	2 per coefficient of each pair: 8t + 8	O(t^2) logic or a single iterative multiplier
Iterations	exactly 2t	data-dependent, t + 1 .. 2t	one per located error position, up to t
Critical path per iteration	one multiply and one XOR, no feedback across the array	cross-multiply feeding a degree comparison that drives the swap: longer	depends on implementation; often dominated by syndrome-update MUX tree
Control	discrepancy select and gamma update	two nominal degree counters, the normalise /	position counter, syndrome-test loop, error-

	inversionless BM ("RIBM")	modified Euclidean ("EUCLID")	step-by-step / PGZ ("SMALL_T")
		cross / swap rule	counter update
Readability	dense; the reformulation is not obvious from the textbook	the textbook algorithm, recognisable step by step	the simplest conceptually for very small t
Best fit	general t; the default if no consumer names a small-t profile	a cross-check and a readable alternative	t <= 8 (flash pages often sit here); smallest area at tiny t

No default is selected at MAS v0.1. The choice is PRD D11.

7.5 Microarchitecture internals

7.5.1 "RIBM" and "EUCLID" — fixed-iteration arrays

Both the reformulated inversionless Berlekamp-Massey array and the modified Euclidean array are pure datapath structures. They need:

- an iteration counter, 0 .. `2 * T_BITS`
- a start pulse from `in_valid && in_ready`
- a completion qualifier when the iteration count reaches its target

Neither implementation has a state machine in the data path. The control signals (discrepancy select, gamma update, degree comparison, swap) are combinational or registered qualifiers derived from the iteration counter and the array registers.

7.5.2 "SMALL_T" — conditional minimal control FSM

The step-by-step and PGZ approaches are control-heavy. If D11 selects `"SMALL_T"`, this block is the one place in the decoder datapath where a minimal control FSM is permitted (per `vault/handbook/design/minimal-fsm.md`). The target state list is the fewest states that carry real distinctions:

```
IDLE   -> wait for in_valid
RUN    -> execute one iteration / error-position test
TEST   -> evaluate the discrepancy / matrix determinant
DONE   -> out_valid raised, hold until out_ready
```

The IDLE/RUN/TEST/DONE list is conditional on `KES_ALGO == "SMALL_T"` and is marked as such in the RTL. States that only wait one fixed cycle are merged into RUN with a sub-counter.

7.6 FSM policy

- For `"RIBM"` and `"EUCLID"`: **no FSM** in the data path; only an iteration counter and completion qualifier.
- For `"SMALL_T"`: **one minimal control FSM** is permitted, conditional on D11, with the state list above.

7.7 Timing

- `"RIBM"`: exactly `2 * T_BITS` iterations.
- `"EUCLID"`: between `T_BITS + 1` and `2 * T_BITS` iterations, data-dependent.
- `"SMALL_T"`: up to `T_BITS` error-position tests, plus setup cycles.
- Exact cycle counts are placeholders tied to PRD D11 and D6.

7.8 Notes

- Two solvers agreeing as a cross-check is the recommended verification posture from the HAS: the golden model runs every candidate on every block and requires identical roots, and the RTL is built with whichever algorithm the first consumer prefers.
- The locator degree `out_lambda_degree` is produced by the solver and is used by the Chien search to determine correctability.
- `Lambda_0` is always 1 for a valid locator; the formal targets include this invariant and the relationship to the odd syndromes.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

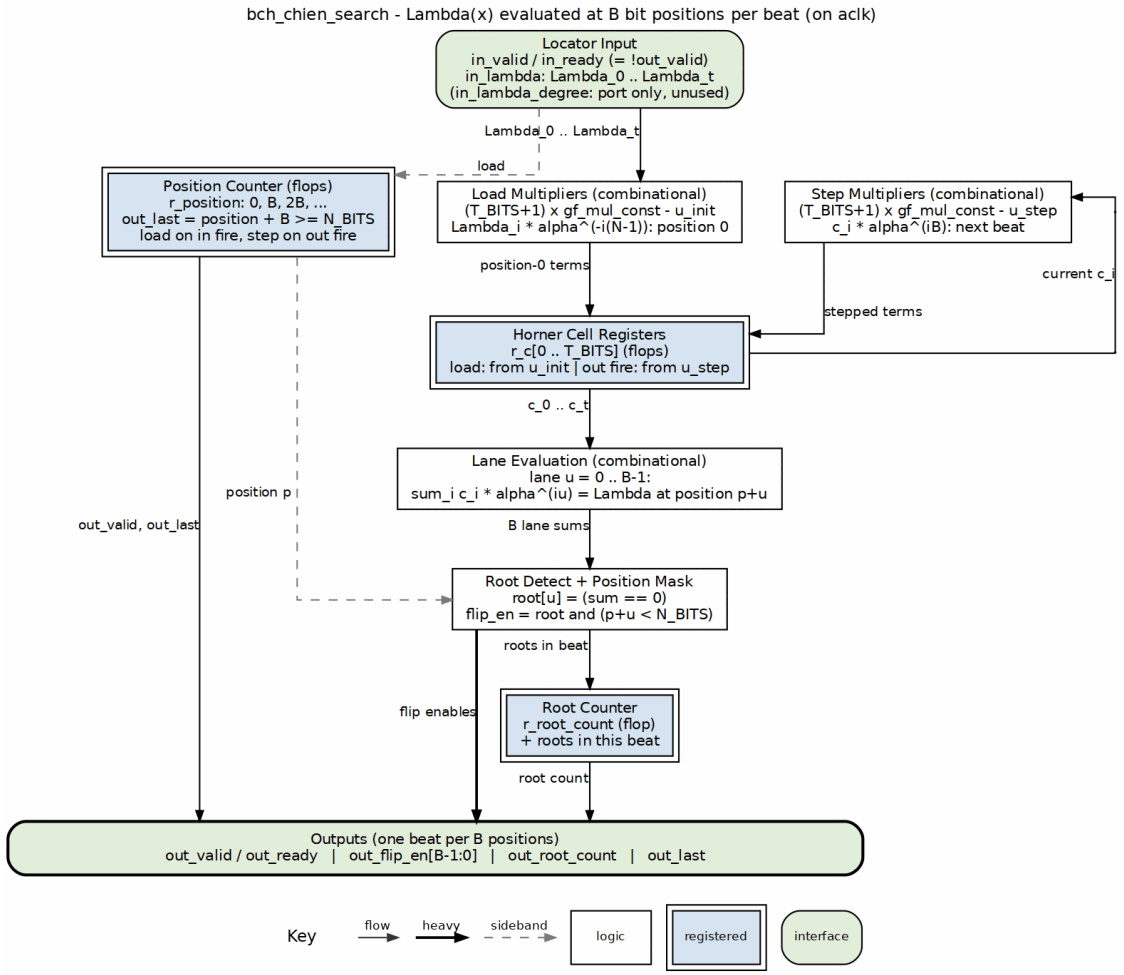
8 Chien Search (bch_chien_search)

Module: bch_chien_search.sv **Location:** rtl/fub/ **Category:** streaming datapath **Parent:** bch_decoder_core **Status:** landed — rtl/fub/bch_chien_search.sv. The flip-enable qualification below is resolved: out_flip_en carries the raw root flags per beat; the w_correctable && !w_release_passthrough qualification is a decoder-core output function (release-on-verdict), applied where the verdict is final, not in this walk.

8.1 Purpose

`bch_chien_search` evaluates the error-locator polynomial `Lambda(x)` at every one of the `N_BITS` bit positions of the received codeword. Where `Lambda` has a root, the corresponding bit is in error. Because the code is binary, every error value is 1 (PRD §2); the block therefore flips the bit directly and needs no Forney evaluator. The block also feeds the uncorrectability verdict by comparing the root count to the locator degree.

8.1.1 Figure 2.3: Chien search block diagram



Chien search block diagram

8.2 Parameters

Table 2.8: Chien search parameters

Parameter	Type	Range	Default	Meaning	PRD
FIELD_DIM	int	3..16	**TBD**	m; field is GF(2^m)	D1
T_BITS	int	1..(2^m-1)/2	**TBD**	correctable bit errors	D2
N_BITS	int	2t+1 .. 2^m-1	**TBD**	codeword length in bits	D3
BITS_PER_BEAT	int	1 .. K_BITS	**TBD**	bits per valid/ready beat	D9

8.3 Interface

Table 2.9: Chien search ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock
`aresetn`	in	1	active-low synchronous reset
`in_valid`	in	1	`Lambda(x)` coefficients are valid
`in_ready`	out	1	unit accepts `Lambda`
`in_lambda`	in	`(T_BITS + 1) * m`	locator coefficients `Lambda_0 .. Lambda_T_BITS`
`in_lambda_degree`	in	`$clog2(T_BITS + 1)`	actual degree of `Lambda`
`out_valid`	out	1	a beat of correction flags is offered
`out_ready`	in	1	downstream accepts correction flags
`out_flip_en`	out	`BITS_PER_BEAT`	one-hot bit-flip enable per position in the beat
`out_root_count`	out	`$clog2(T_BITS + 1)`	roots found so far
`out_last`	out	1	final position of the block

8.4 Microarchitecture internals

8.4.1 Parallel Horner cells

`Lambda(x)` is evaluated at consecutive powers of `alpha` using Horner cells. For position `i`, the evaluation is:

```
Lambda(alpha^-i) = Lambda_0 + Lambda_1*alpha^-i + Lambda_2*alpha^-2i +
... + Lambda_t*alpha^-ti
```

Each cell holds a running partial sum and multiplies by a constant factor each cycle, stepping from one position to the next. With `BITS_PER_BEAT = B > 1`, each lane evaluates `B` positions per cycle by chaining `B` constant multiplies from the same starting register.

The root flag per position is:

```
w_root[i] = (w_lambda_at_pos[i] == 0)
```

8.4.2 Direct bit-flip accumulation

At this fub level `out_flip_en[i]` is the raw root flag for position `i` (gated only by position validity in the final partial beat). The qualifier `w_root[i] && w_correctable && ! w_release_passthrough` lives in `bch_decoder_core`, which owns `w_correctable` (root count matches locator degree, degree within `T_BITS`) and `w_release_passthrough` (uncorrectable and frame-err blocks pass through unchanged, R2). There is no error-value computation anywhere; every root means flip.

8.4.3 Root count (the verdict lives in the decoder core)

The block keeps a running root count, `r_root_count`, and presents `out_root_count = r_root_count + roots in this beat` on every beat; on the last beat it is the block's total. The block makes no verdict of its own. `bch_decoder_core` compares the count with the locator degree it captured from the solver:

```
correctable = !more_than_t && (root_count == lambda_degree) &&
recheck_zero
```

The `in_lambda_degree` port is declared but not read inside this block; the decoder core uses its own registered copy of the degree.

8.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). The Chien walk is driven by:

- `r_position`: counts bit positions, 0 .. `N_BITS-1`
- `r_root_count`: running count of roots
- `out_valid`: registered pipeline flag
- `w_last`: combinational qualifier for the final position

The position counter replaces any state machine.

8.6 Timing

- Serial case (`BITS_PER_BEAT = 1`): `N_BITS` cycles per block to evaluate all positions.
- Parallel-beat case: `ceil(N_BITS / BITS_PER_BEAT)` cycles per block, with `B` Horner evaluations per lane per cycle.
- The exact cycle count and throughput trade-off are placeholders tied to PRD D6.

8.7 Notes

- A shortened code (`N_BITS < 2^m - 1`) is handled by starting the Chien walk at the appropriate offset so that the first emitted position corresponds to bit 0 of the shortened codeword.
- The no-Forney structure is a key formal target: prove that every root of `Lambda` in a correctable block corresponds to exactly one bit flip and that the resulting corrected block has zero syndrome.
- `out_flip_en` is a beat-wide vector; the corrector XORs it with the corresponding beat from the block buffer.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

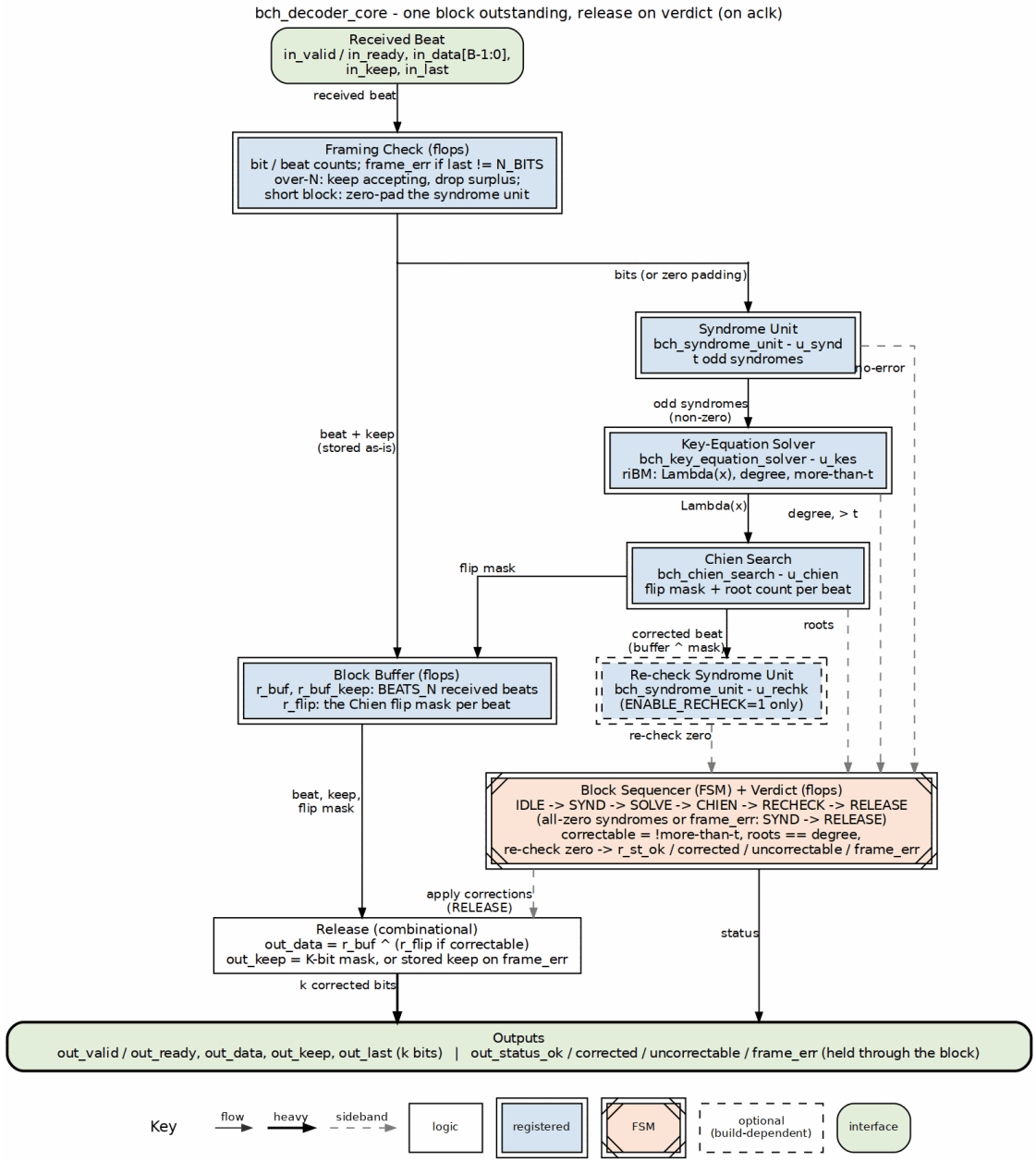
9 Decoder Core Integration (bch_decoder_core)

Module: bch_decoder_core.sv **Location:** rtl/macro/ **Category:** integration block + one minimal control FSM **Parent:** bch_top / standalone decoder wrapper **Status:** landed — rtl/macro/bch_decoder_core.sv. The sequencer has a sixth state RECHECK (IDLE/SYND/SOLVE/CHIEN/RECHECK/RELEASE): the verdict is computed only after the optional re-check syndrome unit finishes, because the re-check result is not known until the corrected stream's last beat. Corrections are applied at the output read, not during the Chien walk (release-on-verdict, mirroring rs_decoder_core), so the MAS timing note "first corrected beat leaves as soon as the Chien walk starts" is corrected: the corrected image is computed during the walk but released only after the verdict. Single outstanding block; pipelining is PRD D6 open work. Framing violations cannot deadlock the input (issue #90): a block that reaches N_BITS without in_last, or overshoots it, is buffered up to BEATS_N beats, released with frame_err, and the syndrome unit is cleared for the next block.

9.1 Purpose

`bch_decoder_core` integrates the received-block buffer, syndrome unit, key-equation solver, Chien search, and corrector into one valid/ready streaming block. It is the only block in the decoder datapath that is allowed to contain a control FSM, because the sequencing of distinct algorithmic stages is genuine control, not per-beat datapath.

9.1.1 Figure 2.4: Decoder core block diagram



Decoder core block diagram

9.2 Parameters

Table 2.10: Decoder core parameters

Parameter	Type	Range	Default	Meaning	PRD
FIELD_DIM	int	3..16	**TBD**	m; field is GF(2^m)	D1
T_BITS	int	1..(2^m-1)/2	**TBD**	correctable bit errors	D2
N_BITS	int	2t+1 .. 2^m-1	**TBD**	codeword length in bits	D3
BITS_PER_BEAT	int	1 .. K_BITS	**TBD**	bits per valid/ready beat	D9
PRIM_POLY	int	irreducible of degree m	**TBD**	primitive polynomial	D8
FIRST_ROOT	int	0 .. 2^m-2	**TBD**	b, first root of g(x)	D8
K_BITS	int	derived	N_BITS - deg g(x)	message bits; exposed for convenience, do not override	–
ENABLE_RECHECK	bit	0, 1	1	run a second syndrome unit over the corrected stream and fold its zero-syndrome result into	–

Parameter	Type	Range	Default	Meaning	PRD
				the verdict	

`bch_decoder_core` has no `ENABLE_ERASURES` or `KES_ALGO` parameter. Erasure decoding is not implemented (PRD D5 is open, and the imported solver is instantiated with `ERASURE_SUPPORT = 0`). `KES_ALGO` is a parameter of `bch_key_equation_solver` only, left at its default `"RIBM"` by the core; any other value is an elaboration error (Section 2.3).

9.3 Interface

Table 2.11: Decoder core ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock
`aresetn`	in	1	active-low synchronous reset
`in_valid` / `in_ready`	in / out	1	valid/ready for received bits
`in_data`	in	`BITS_PER_BEAT`	received bits
`in_keep`	in	`BITS_PER_BEAT`	present-bit mask
`in_last`	in	1	this beat carries the `N_BITS`-th received bit
`out_valid` / `out_ready`	out / in	1	valid/ready for corrected data
`out_data`	out	`BITS_PER_BEAT`	corrected data bits; parity lanes dropped
`out_keep`	out	`BITS_PER_BEAT`	present-bit mask
`out_last`	out	1	this beat carries the `K_BITS`-th data bit
`out_status_ok`	out	1	block had no errors
`out_status_corrected`	out	`$clog2(T_BITS + 1)`	bits corrected (0 .. `T_BITS`)
`out_status_uncorrectable`	out	1	correction failed
`out_status_frame_err`	out	1	block length was not `N_BITS`

9.4 Microarchitecture internals

9.4.1 Block buffer

The received block is written into a register buffer as it arrives: `r_buf` and `r_buf_keep` hold `BEATS_N = ceil(N_BITS / BITS_PER_BEAT)` beats, and `r_flip` holds the Chien flip mask for each beat, written during CHIEN. The buffer is read out in RELEASE, after the verdict. For an uncorrectable block, the buffer passes the received bits through unchanged (R2):

```
w_release_passthrough = w_uncorrectable || w_frame_err
```

9.4.2 Stage sequencer

The decoder core contains exactly one minimal control FSM (per `vault/handbook/design/minimal-fsm.md`). Stage sequencing is control, not per-beat datapath. The proposed state list merges any state that only waits one fixed cycle into its successor with a counter:

```
IDLE    -> wait for a new block to begin arriving
SYND    -> receiving the block into the buffer and the syndrome unit
SOLVE   -> key-equation solver running
CHIEN   -> Chien search; flip masks stored beside the buffered beats
RECHECK -> wait for the re-check syndrome unit (skipped if
ENABLE_RECHECK=0)
RELEASE -> outputting the corrected (or passthrough) block
```

A state like “wait one cycle for solver start” is killed: it becomes a registered `r_solver_start` pulse inside SYND or SOLVE.

9.4.3 Corrected-stream re-check

As an R2 mechanism, a second syndrome unit (`u_rechk`, present when `ENABLE_RECHECK = 1`, the default) accumulates over the corrected stream, `r_buf ^ flip mask`, beat by beat during CHIEN. If any odd syndrome is non-zero after correction, the block is flagged uncorrectable even if the Chien root count matched the locator degree. This mirrors the release-on-verdict discipline in the sibling `rs_decoder_core` architecture. The re-check is gated by a build parameter and is an implementation detail, not part of the locator search.

9.4.4 Correctability verdict

The per-block uncorrectable verdict is:

```
w_uncorrectable = !w_correctable || (w_recheck_enabled && !
w_recheck_zero)
w_correctable = (r_root_count == r_lambda_degree) && (r_lambda_degree
<= T_BITS)
```

`out_status_uncorrectable` is held for every beat of the released block.

9.5 FSM policy

This block is the **one place in the decoder datapath where a minimal control FSM is permitted**. The FSM has the six states above; any state that only waits one fixed cycle is merged with its successor. All per-beat operations inside the stages (syndrome accumulation, solver array stepping, Chien evaluation) remain FSM-free pipelines.

9.6 Timing

- `SYND` phase: `ceil(N_BITS / BITS_PER_BEAT)` beats.
- `SOLVE` phase: depends on `KES_ALGO` and `T_BITS` (PRD D11).
- `CHIEN` / `RELEASE` phase: `ceil(N_BITS / BITS_PER_BEAT)` beats, overlapped so that the first corrected beat leaves as soon as the Chien walk starts.
- Total latency and throughput trade-offs are placeholders tied to PRD D6.

9.7 Notes

- This whole page is conditional on the build including the decoder (PRD D4) and on the throughput architecture (PRD D6) and solver choice (PRD D11).
- Frame error: a block whose `in_last` arrives before or after the `N_BITS`-th bit is flagged with `out_status_frame_err` and passes through unchanged. The first `length - (N_BITS - K_BITS)` bits of a too-long block are treated as data.
- Status outputs describe the block being emitted and are valid on every beat of it; a consumer may sample them with `out_last`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

10 Core Signals

This page presents the valid/ready core signals as a contract table rather than restating the prose interface description in `../bch_has/ch04_interfaces/01_core_interface.md`. The direction, reset value, and creator block for each signal are the source of truth here.

10.1 Reset strategy

All core logic uses **synchronous active-low reset** (`aresetn`), the house clocking convention. Reset clears block state, counters, the LFSR, syndrome registers, solver arrays, and the Chien position counter. The block buffer itself is not reset; its read/write pointers are reset, which makes the buffer contents logically invalid until overwritten.

10.2 bch_encoder_core signal contract

Table 3.1: bch_encoder_core signal contract

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
aclk	in	1	—	clock source	one clock for the whole core	—
aresetn	in	1	—	reset source	synchronous active-low reset	—
in_valid	in	1	—	upstream	stable until in_ready high	encoder core
in_ready	out	1	0	encoder core	combinational ! r_valid || m_ready gated by parity-drain	encoder core
in_data	in	BITS_PER_BEAT	—	upstream	data bits, low-aligned; partial final beat qualified by in_keep	encoder core
in_keep	in	BITS_PER_BEAT	—	upstream	present-bit mask, low-aligned;	encoder core

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
					partial only on final beat	
in_last	in	1	—	upstream	this beat carries the K_BITS-th data bit	encoder core
out_valid	out	1	0	encoder core	registered pipeline flag; data beats then parity beats	encoder core
out_ready	in	1	—	downstream	may depend on out_valid	encoder core
out_data	out	BITS_PER_BEAT	0	encoder core	data while !w_parity_phase, parity while w_parity_phase	encoder core
out_keep	out	BITS_PER_BEAT	0	encoder core	full except on final data and final parity beats	encoder core
out_last	out	1	0	encoder	this beat	encoder

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
				core	carries the N_BITS-th coded bit	core
frame_err	out	1	0	encoder core	pulse when in_last does not coincide with K_BITS data bits	encoder core

10.3 bch_decoder_core signal contract

Table 3.2: bch_decoder_core signal contract

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
in_valid	in	1	—	upstream	stable until in_ready high	decoder core
in_ready	out	1	0	decoder core	high while buffer has space; drops when full	decoder core
in_data	in	BITS_PER_BEAT	—	upstream	received bits, low-aligned	decoder core
in_keep	in	BITS_PER_BEAT	—	upstream	present-bit mask; partial only on final beat	decoder core
in_last	in	1	—	upstream	this beat carries the N_BITS-th received bit	decoder core
in_erase	in	BITS_PER_BEAT	—	upstream	(only with ENABLE_ERASURES) known-bad bits	decoder core

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
out_valid	out	1	0	decoder core	high once the block verdict is final	decoder core
out_ready	in	1	—	downstream	may depend on out_valid	decoder core
out_data	out	BITS_PER_BEAT	0	decoder core	corrected data bits; parity lanes dropped	decoder core
out_keep	out	BITS_PER_BEAT	0	decoder core	present-bit mask; partial only on final beat	decoder core
out_last	out	1	0	decoder core	this beat carries the K_BITS-th data bit	decoder core
out_status_ok	out	1	0	decoder core	held for every beat: block had no errors	syndrome unit, decoder core
out_status_corrected	out	$clog2(T_BITS+1)	0	decoder core	held for every beat: bits corrected (0 .. T_BITS)	Chien search, decoder core

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
`out_status_uncorrectable`	out	1	0	decoder core	held for every beat: correction failed; data pass through unchanged	Chien search, decoder core
`out_status_frame_err`	out	1	0	decoder core	held for every beat: block length was not `N_BITS`	decoder core

10.4 Timing rules

- `in_last` is the sole block boundary; the cores do not infer boundaries by counting.
- `out_status_*` describe the block being emitted and are valid on every beat of it, first to last. A consumer may sample them with `out_last` or with the first beat.
- An uncorrectable block leaves exactly as received; the corrector is disabled by `w_release_passthrough`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

11 Signal Contracts and K-maps

11.1 What a contract is

A signal contract in this repo is a machine-checkable statement of what a signal is allowed to do. Each contract has three parts, in order:

1. **Term list:** every signal or expression in play, with its defining expression and `file:line` citation.
2. **Invariants:** the strict relationships between terms, each cited, each stating which rows of the decision table it renders impossible.
3. **Decision table:** one row per combination of the terms. Rows the invariants forbid are marked `ILLEGAL`; legal rows carry the resulting output.

The canonical methodology is in `vault/handbook/design/signal-contracts-and-kmaps.md`; the shared generator machinery is in `bin/kmaps/`.

11.2 Which signals have contracts

The BCH workbook documents the control/qualifier signals whose correctness is non-obvious: parity-phase select, block-boundary and frame-error qualifiers, syndrome completion, no-error bypass, correctability verdict, Chien root-position enables, and uncorrectable pass-through. The complete list is in the generated workbook.

11.3 Pre-RTL citation posture

At MAS v0.1 there is no RTL, so the contracts cite the MAS pages where the intended expressions are written verbatim. When the first RTL lands, every MAS citation is re-pointed at the corresponding `.sv` file and line. The generator's citation gate then enforces that the workbook and the RTL agree; a drift fails the run.

11.4 Regenerating the workbook

```
python3
projects/components/ecc-ip/bch/docs/gen_bch_signal_contracts_kmaps.py
```

The generator writes:

```
projects/components/ecc-ip/bch/docs/bch_signal_contracts.xlsx
```

Do not hand-edit the `.xlsx`; it is a build artifact of the generator.

11.5 Workbook contents

The generated workbook contains:

- a contract sheet for the core signals
- a kmap sheet per block with the key combinational decisions
- term lists, invariants, decision tables, `depends_only_on` sufficiency arguments, and `rtl_sop` expressions for review against the landed RTL

Since the 2026-10-08 citation migration (TASK-003 close) every citation points at the landed RTL `file:line` and the generator's `verify_citations` gate fails the build if an RTL edit moves a quoted line. For every kmap the verdict is the derived minimal cover diffed against `rtl_sop` (IDENTICAL or DIFFERS); all eight maps of the landed design render IDENTICAL.